
14. Mutal Exclusion

상호 배제를 제공하는 데에는 binary semaphore 가 사용된다 . 그러나 semaphore 가 캡슐화된 경우에는 태스크가 자원에 접근할 때 semaphore 의 획득에 관하여 알지 못할 수 있다 .

Counting semaphore 는 보다 일반적인 형태로서 , 자원이 동시에 하나 이상의 태스크에 의하여 사용될 때 사용될 수 있다 .

Real-time kernel 의 경우 코드의 critical section 에 들어가면 인터럽트를 disable 시킨다.

예를 들어서 두개의 task가 동시에 printer를 쓰려고 한다고 가정하자. 만약 task1이 프린트 하는 도중에 task2가 실행되고 task2가 실행되는 도중에 다시 task1이 실행된다면 나온 결과는 task1이나 task2모두 원하지 않는 결과일 것이다. 이때 mutual exclusion이 되어야 할 필요가 있다. 또한 위에서 언급한 shared memory도 마찬가지이다. 즉 shared resource를 쓰는 경우에는 mutual exclusion이 보장되어야 하는 것이다.

이를 해결하기 위해서는 몇가지 방법이 있는데 첫째 가장 쉬운 방법은 interrupt가 발생하는 것을 막는 것이다. 어떤 task에서 다른 task로 제어권이 넘어가는 경우의 한가지로 자신이 message를 기다리는 상태로 가는 상황을 생각해 볼 수 있는데 이런 것을 shared resource를 쓰는 부분에서는 제외시킬 수 있다. 이런 상황을 막는다고 하면 남는 것은 round-robin에 의해서 다른 task로 제어권이 넘어가는 경우나 interrupt가 발생하는 경우다. Round-robin에서 다른 task로 제어권을 넘기는 것 역시 timer(clock) interrupt가 발생한 경우이므로 한 마디로 interrupt를 막으면 된다. 즉, critical section으로 들어가기 전에 interrupt를 disable시키고 빠져 나오면서 interrupt를 enable시킨다.

이외에 semaphore라는 mechanism이 이용된다.(semaphore에 대해서는 아래에서 설명)

semaphore를 얻지 못하면 shared resource를 얻을 수 없으며 일단 semaphore를 얻으면 shared resource를 마음 놓고 쓸 수 있다. 다른

task를 위해서 shared resource를 다 쓰면 semaphore를 풀어야 한다.

15. Semaphores

Semaphore 는 다음과 같은 용도를 갖는다 .

- 공유된 자원의 사용을 제어한다 . (상호배제 : mutual exclusion)
- event 의 발생을 알린다 .
- 두 태스크의 동작을 동기화한다 .

uC/OS 는 binary semaphore 와 counting semaphore 를 모두 지원한다

.

Semaphore 와 연관된 연산은 다음과 같은 것이 있다 .

INITIALIZE (or CREATE)

Semaphore 의 초기값을 준다 . 처음 waiting list 는 비어 있다 .

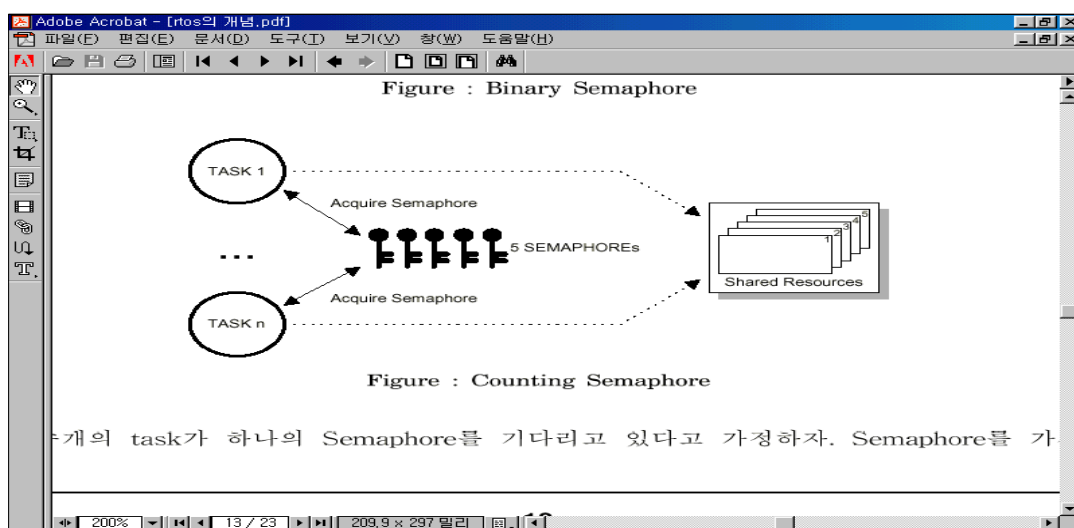
WAIT (or PEND)

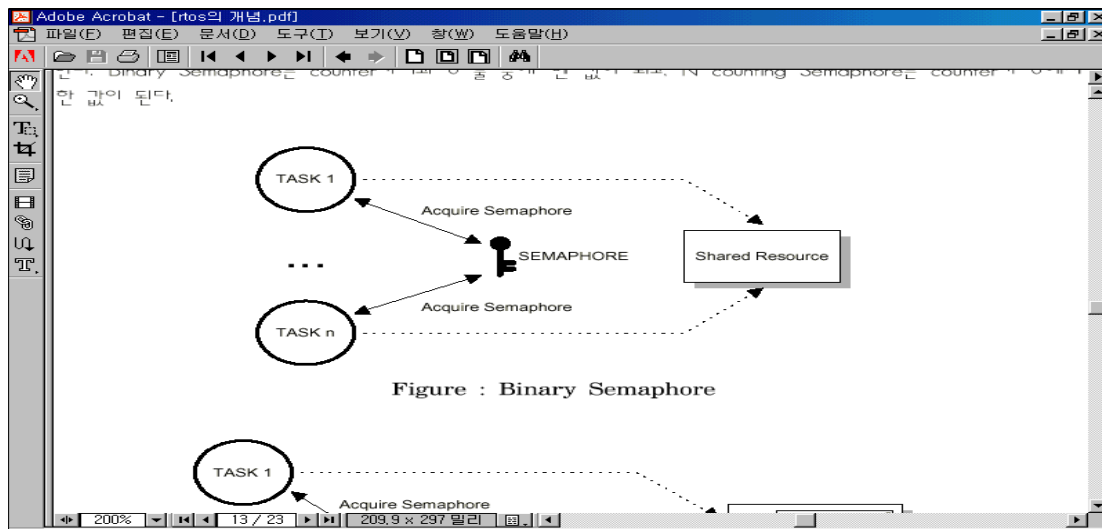
Semaphore 를 요구하는 태스크가 WAIT 연산을 행하면 semaphore 값이 감소하다가 0 이되면 해당 태스크는 waiting list 에 들어가게 된다 .

SIGNAL (or POST)

태스크는 이 연산을 사용하여 semaphore 를 놓게 된다 .

uC/OS 에서는 semaphore 를 기다리는 태스크 중 가장 높은 priority 를 갖는 것이 semaphore 를 갖도록 한다.





16. Deadlock

deadlock은 시스템이 더 이상 진행될 수 없는 상태에 도달하는 것을 말한다

17. Synchronization

semaphore 를 사용하여 어떤 태스크를 ISR 또는 다른 태스크와 동기화시킬 수 있다 . 이 경우에 semaphore 는 상호 배제를 위하여 사용된 것이 아니라 이벤트의 발생을 알리기 위한 플래그로서 사용된 것이다 . 이렇게 semaphore 를 사용하는 것을 unilateral rendezvous 라고도한다 .

이 경우 semaphore 의 초기값은 0 이다 . 예를 들어 태스크가 I/O 연산을 시작하고 semaphore를 기다릴 때 I/O 연산이 끝나면 ISR 이나 다른 태스크가 semaphore로 신호를 보내고 태스크가 계속 진행되게 된다 .

18. Event Flags

Event flag는 Event가 여러개 발생했을 때의 synchronize를 위해서 필요하다. Nucleus에서는 Event group이라는 용어로 쓰인다. 이

group에서는 한 bit마다 특정 event에 대응해서 어떤 event가 발생되면 그 bit(event flag)가 set되며 이때 그 event를 기다리는 task가 ready상태가 된다.

19. Message Mailboxes

이것은 message exchange 라고도 하며 , 대개 pointer size 변수이다

커널에 의하여 제공되는 서비스를 사용하여 태스크나 ISR 이 메시지 (포인터)를 메일박스에 보내게 되며 하나 이상의 태스크가 이 메시지를 받을 수 있다 .

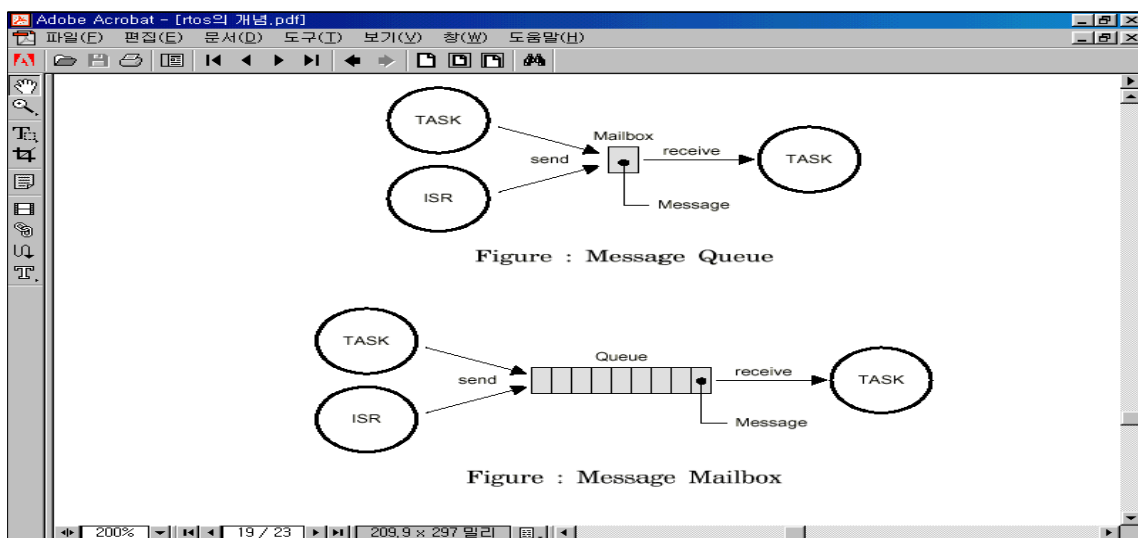
메시지가 메일박스에 놓여지게 되면 uC/OS 에서는 메시지를 기다리는 태스크 중 가장 높은 우선 순위를 가지는 태스크에 메시지를 전달하게 된다.

20. Message Queues

Message queue 는 메일박스의 배열로FIFO 형태로 되어 있다 .

비어 있는 큐로부터 메시지를 받으려고 하는 태스크는 서스펜드되어 메시지가 올 때까지 waiting list 에 놓여지게 된다 .

uC/OS 에서는 메시지를 기다리는 태스크 중 가장 높은 우선 순위를 가지는 태스크에게 메시지를 전달한다.



21. Interrupts

인터럽트 발생이 확인되면 CPU 는 context 를 저장하고 Interrupt Service Routine(ISR) 서브루틴으로 분기한다 .

ISR 이 완료되면 프로그램은 실행 준비가 된 태스크 중 가장 우선 순위가 높은 것으로 돌아간다 .

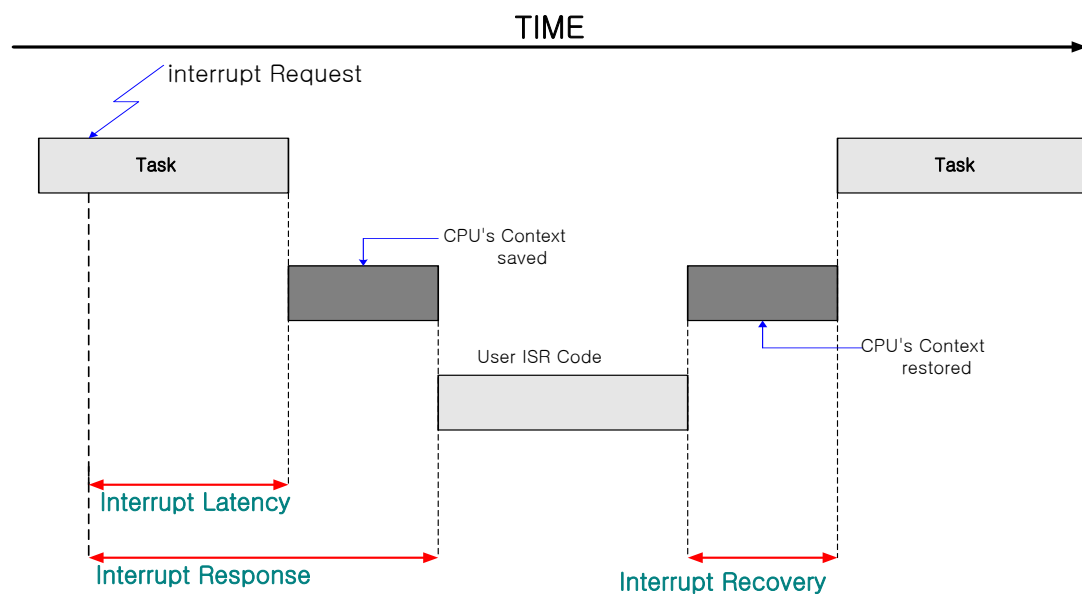
Interrupt Nesting : ISR 진행 도중 다시 interrupt 발생

Interrupt Latency, Response and Recovery

Interrupt Latency : Maximum amount of time

The Response Time :

Interrupt latency + Time to save CPU's context + Time to determine if a higher priority task is ready



22. NMIs(Non-Maskable Interrupts)

인터럽트 중 최대한 빨리 서비스되어야하는 것이나 커널에 의한 latency 를 가질 수 없는 것의 경우에 해당된다 . 이것은 disabled 될 수 없으며 interrupt latency 나 response, recovery 타임이 최소이다 .

ex) saving important information during a power down

Parameters : 전역변수와 이들 변수들의 크기가 한꺼번에 read/write 되어야 한다.

23. Clock Tick

Clock tick 은 주기적으로 발생하는 인터럽트로 일정 수의 tick 동안 태스크가 실행을 중단(suspend)하거나 이벤트의 발생을 기다리도록 한다 . 이것의 주기가 빨라지면 그만큼 오버헤드도 커진다.

tick ISR 은 OSTimeTick()을 호출하는데 이것은 OS_TCB 의 OSTCBDly 값이 nonzero 값이면 감소시킨다 . 이것이 0 이 되면 태스크는 실행될 준비가 된다 .

OSTimeTick()은 태스크 레벨에서 호출될 수 있는데 이렇게 하려면 priority 를 5 로 할당할수 있다 . (priority 0 에서 4 까지는 예약됨)

```
TimeTask()  
while(1 )  
OSSemPend(...);  
OSTimeTick();
```

tick interrupt 가 발생하였다는 것을 알리기 위하여 semaphore(0 으로 초기화)를 생성할 수있다 .

OSTimeTick()은 power updlgn 로 OSTime 이라는 32bit 값을 더해 나가는데 이것은 OSTimeGet()함수를 이용하여 얻어낼 수 있다 . 이 값의 설정은OSTimeSet()함수를 이용한다 .